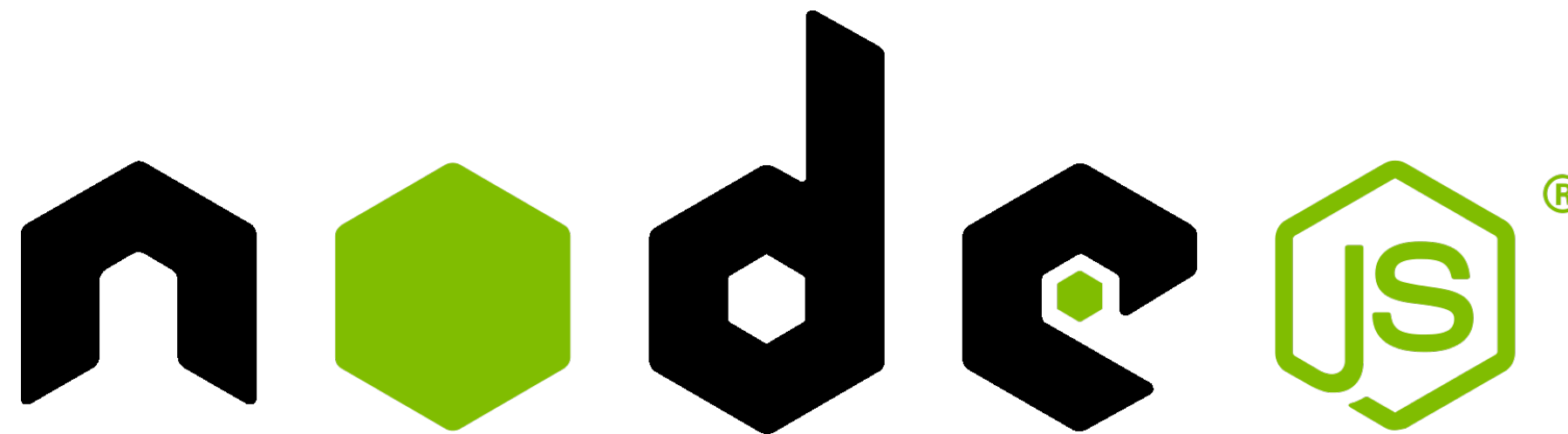

JAVASCRIPT

CLASS, MODULE, ERROR HANDLING

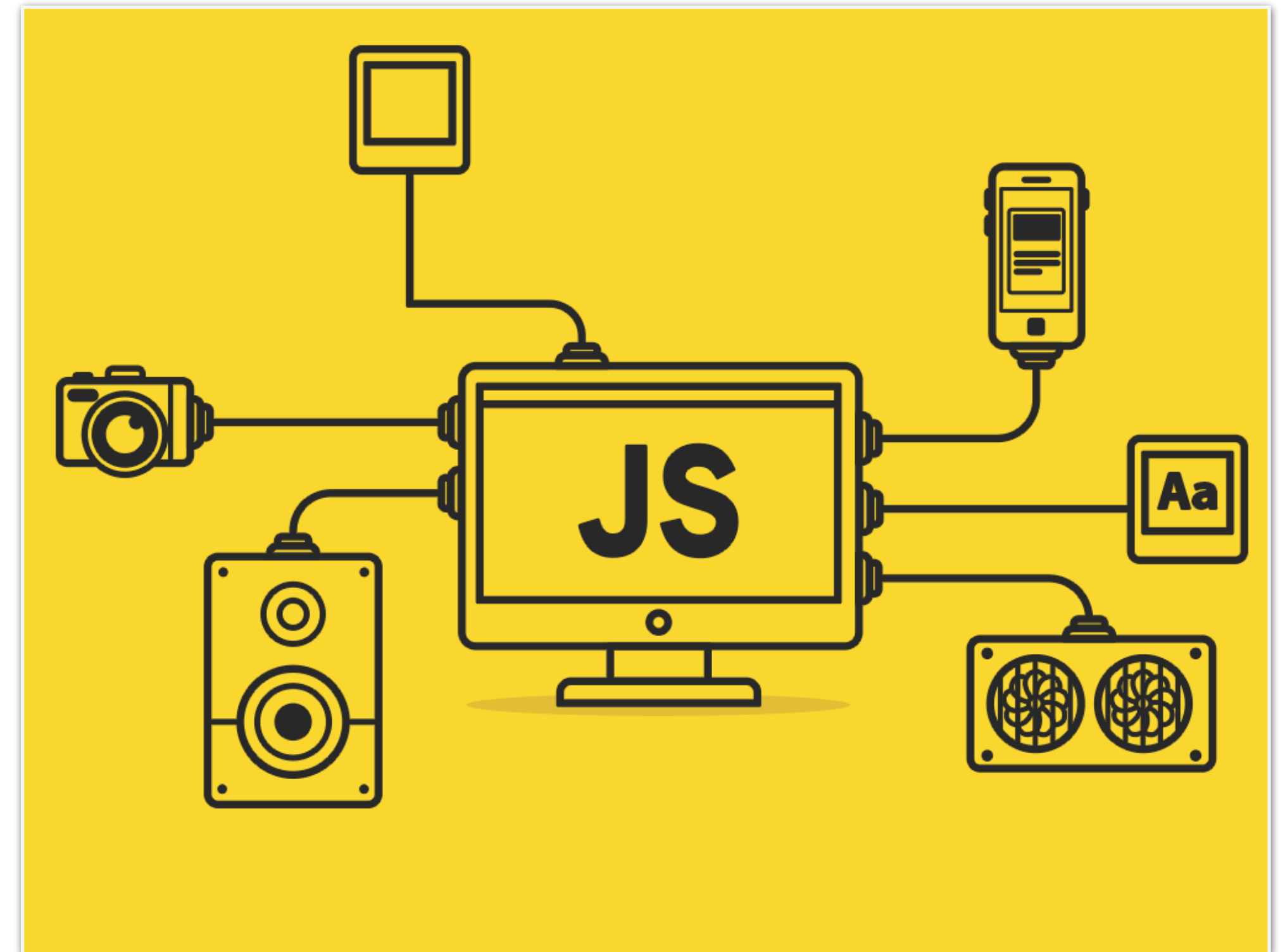
Anirach Mingkhwan

Anirach.m@fitm.kmutnb.ac.th



OUTLINE

- Class
- Module
- Error Handling



CLASS

CLASS SYNTAX

```
1 class MyClass {  
2   // class methods  
3   constructor() { ... }  
4   method1() { ... }  
5   method2() { ... }  
6   method3() { ... }  
7   ...  
8 }
```

Then use `new MyClass()` to create a new object with all the listed methods.

The `constructor()` method is called automatically by `new`, so we can initialize the object there.

CLASS

JavaScript supports the concept of *classes* as a syntax for creating objects. Classes specify the shared properties and methods that objects produced from the class will have.

When an object is created based on the class, the new object is referred to as an *instance* of the class. New instances are created using the `new` keyword.

The code sample shows a class that represents a `Song`. A new object called `mySong` is created underneath and the `.play()` method on the class is called. The result would be the text `Song playing!` printed in the console.

```
class Song {  
  constructor() {  
    this.title;  
    this.author;  
  }  
  
  play() {  
    console.log('Song playing!');  
  }  
}  
  
const mySong = new Song();  
mySong.play();
```

CLASS

Lecture-04-ClassCompatModule > class > JS class.js > ...

```
1  class Song {
2    constructor() {
3      this.title;
4      this.author;
5    }
6
7    play() {
8      console.log("Song playing!");
9    }
10 }
11
12 const mySong = new Song();
13 mySong.play();
14
```

```
> node class.js
Song playing!
```

```
~/C/JavaScript
```


CLASS METHODS

Properties in objects are separated using commas. This is not the case when using the `class` syntax. Methods in classes do not have any separators between them.

```
class Song {  
  play() {  
    console.log('Playing!');  
  }  
  
  stop() {  
    console.log('Stopping!');  
  }  
}
```


CLASS CONSTRUCTOR

Classes can have a `constructor` method. This is a special method that is called when the object is created (instantiated). Constructor methods are usually used to set initial values for the object.

```
class Song {  
  constructor(title, artist) {  
    this.title = title;  
    this.artist = artist;  
  }  
}  
  
const mySong = new Song('Bohemian Rhapsody',  
  'Queen');  
console.log(mySong.title);
```


CLASS CONSTRUCTOR

Lecture-04-ClassCompatModule > class > JS constructorclass.js > ...

```
1  class Song {
2    constructor(title, artist) {
3      this.title = title;
4      this.artist = artist;
5    }
6  }
7
8  const mySong = new Song("Bohemian Rhapsody", "Queen");
9  console.log(mySong.title);
10
```

```
> node constructorclass.js
Bohemian Rhapsody
```

```
~/C/JavaScript/Lec
```

CLASS STATIC METHODS

Within a JavaScript class, the `static` keyword defines a static method for a class. Static methods are not called on individual instances of the class, but are called on the class itself. Therefore, they tend to be general (utility) methods.

```
class Dog {  
  constructor(name) {  
    this._name = name;  
  }  
  
  introduce() {  
    console.log('This is ' + this._name + ' !');  
  }  
  
  // A static method  
  static bark() {  
    console.log('Woof!');  
  }  
}  
  
const myDog = new Dog('Buster');  
myDog.introduce();  
  
// Calling the static method  
Dog.bark();
```

CLASS STATIC METHODS

```
1 class Dog {
2   constructor(name) {
3     this._name = name;
4   }
5
6   introduce() {
7     console.log("This is " + this._name + " !");
8   }
9
10  // A static method
11  static bark() {
12    console.log("Woof!");
13  }
14 }
15
16 const myDog = new Dog("Buster");
17 myDog.introduce();
18
19 // Calling the static method
20 Dog.bark();
```

➤ `node staticmethod.js`
This is Buster !
Woof!



CLASS EXTENDS

JavaScript classes support the concept of inheritance — a child class can *extend* a parent class. This is accomplished by using the `extends` keyword as part of the class definition.

Child classes have access to all of the instance properties and methods of the parent class. They can add their own properties and methods in addition to those. A child class constructor calls the parent class constructor using the `super()` method.

```
// Parent class
class Media {
  constructor(info) {
    this.publishDate = info.publishDate;
    this.name = info.name;
  }
}

// Child class
class Song extends Media {
  constructor(songData) {
    super(songData);
    this.artist = songData.artist;
  }
}

const mySong = new Song({
  artist: 'Queen',
  name: 'Bohemian Rhapsody',
  publishDate: 1975
});
```


CLASS EXTENDS

Lecture-04-ClassCompatModule > class > JS extendclass.js

```
1  // Parent class
2  class Media {
3    constructor(info) {
4      this.publishDate = info.publishDate;
5      this.name = info.name;
6    }
7  }
8
9  // Child class
10 class Song extends Media {
11   constructor(songData) {
12     super(songData);
13     this.artist = songData.artist;
14   }
15 }
16
```

```
17 const mySong = new Song({
18   artist: "Queen",
19   name: "Bohemian Rhapsody",
20   publishDate: 1975,
21 });
22
23 console.log(mySong);
```

```
> node extendclass.js
Song { publishDate: 1975, name: 'Bohemian Rhapsody', artist: 'Queen' }
```

```
🍏 > ~/C/JavaScript/Lecture-04/class > on P master !1 ?2
>
```

MODULES

IMPORT JAVASCRIPT MODULES WITH THE REQUIRE FUNCTION

In Node.js, the `require` function can be used to import code from another file into the current script.

```
var moduleA = require( "./module-a.js" );

// The .js extension is optional
var moduleA = require( "./module-a" );
// Both ways will produce the same result.

// Now the functionality of moduleA can be used
console.log(moduleA.someFunctionality)
```


JAVASCRIPT EXPORT DEFAULT

As of ES6, the *export default* keywords allow for a single variable or function to be exported, then, in another script, it will be straightforward to import the default export.

After using the *export default* it is possible to import a variable or function without using the `require()` function.

```
// module "moduleA.js"
export default function cube(x) {
  return x * x * x;
}

// In main.js
import cube from './moduleA.js';
// Now the `cube` function can be used
straightforwardly.
console.log(cube(3)); // 27
```

INTERMEDIATE JAVASCRIPT: EXPORT MODULE

To make an object in our Javascript file exportable as a module in Node.js, we assign the object to the `exports` property of `module`.

```
let Course = {};  
Course.name = "Javascript Node.js"  
module.exports = Course;
```

USING THE IMPORT KEYWORD IN JAVASCRIPT

As of ES6, the `import` keyword can be used to import functions, objects or primitives previously exported into the current script.

There are many ways to use the `import` keyword, for example, you can import all the exports from a script by using the `*` selector as follows: `import * from 'module_name';`.

A single function can be imported with curly brackets as follows: `import {funcA} as name from 'module_name';`

Or many functions by name: `import {funcA, funcB} as name from 'module_name';`

```
// add.js
export const add = (x, y) => {
    return x + y
}
```

```
// main.js
import { add } from './add';
console.log(add(2, 3)); // 5
```


MODULE DEMO

Lecture-04-ClassCompatModule > modules > JS libHello.js > ...

```
1 // libHello.js
2 const person = {
3   name: 'Anirach',
4   weight: 85
5 }
6
7 function sayHello(){
8   console.log("Hello World!")
9 }
10
11 function cube(x){
12   return x*x*x;
13 }
14
15 const add = (a,b) => a+b;
16
17 const status = true;
18
19 module.exports = {person, sayHello, cube, add, status}
```

Lecture-04-ClassCompatModule > modules > JS mainHello.js > ...

```
1 const hello = require('./libHello')
2
3 hello.sayHello();
4 console.log(hello.person.name);
5 console.log(hello.cube(3));
6 console.log(hello.add(5,4))
7 console.log(hello.status)
8
```

> node mainHello.js

Hello World!

Anirach

27

9

true

MODULE DEMO

Lecture-04-ClassCompatModule > modules > JS libArea.mjs > ...

```
1  // libArea.mjs
2  function areaTriangle(b,h) {
3    |    return (b*h)/2;
4  }
5
6  function areaCircle(r){
7    |    return Math.PI * r*r;
8  }
9
10 function areaSqr(w,l){
11   |    return w*l;
12 }
13
14 export { areaCircle, areaTriangle, areaSqr };
```

Lecture-04-ClassCompatModule > modules > JS mainArea.mjs

```
1  import { areaCircle,areaSqr } from "./libArea.mjs";
2
3  console.log(areaCircle(6));
4  console.log(areaSqr(3,4));
5
```

```
> node mainArea.mjs
113.09733552923255
12
```

*.mjs = Authors can tell Node.js to treat JavaScript code as ECMAScript modules via the *.mjs file extension

ERROR HANDLING

UNINTENDED BEHAVIOR

Exception

Interruption in the intended execution of code

Error

Unintended interruption in execution of code, typically show up as exceptions

Syntax errors are a type of exception that occurs when incorrect syntax is used

Throwing an Exception

Sending a message that something has gone wrong in the intended execution of code, this can be thrown by JavaScript or by the developer with the `throw` keyword

HANDLING ERROR IN JAVASCRIPT

Uncaught Exceptions

Exceptions that have been thrown when code has no guidance to handle the interruption

Users may see unhelpful obscure messages, or none exist

Catch exceptions with `try..catch..finally`

Try

Block of code that may throw an exception

Catch

Block of code that will run if an exception is thrown

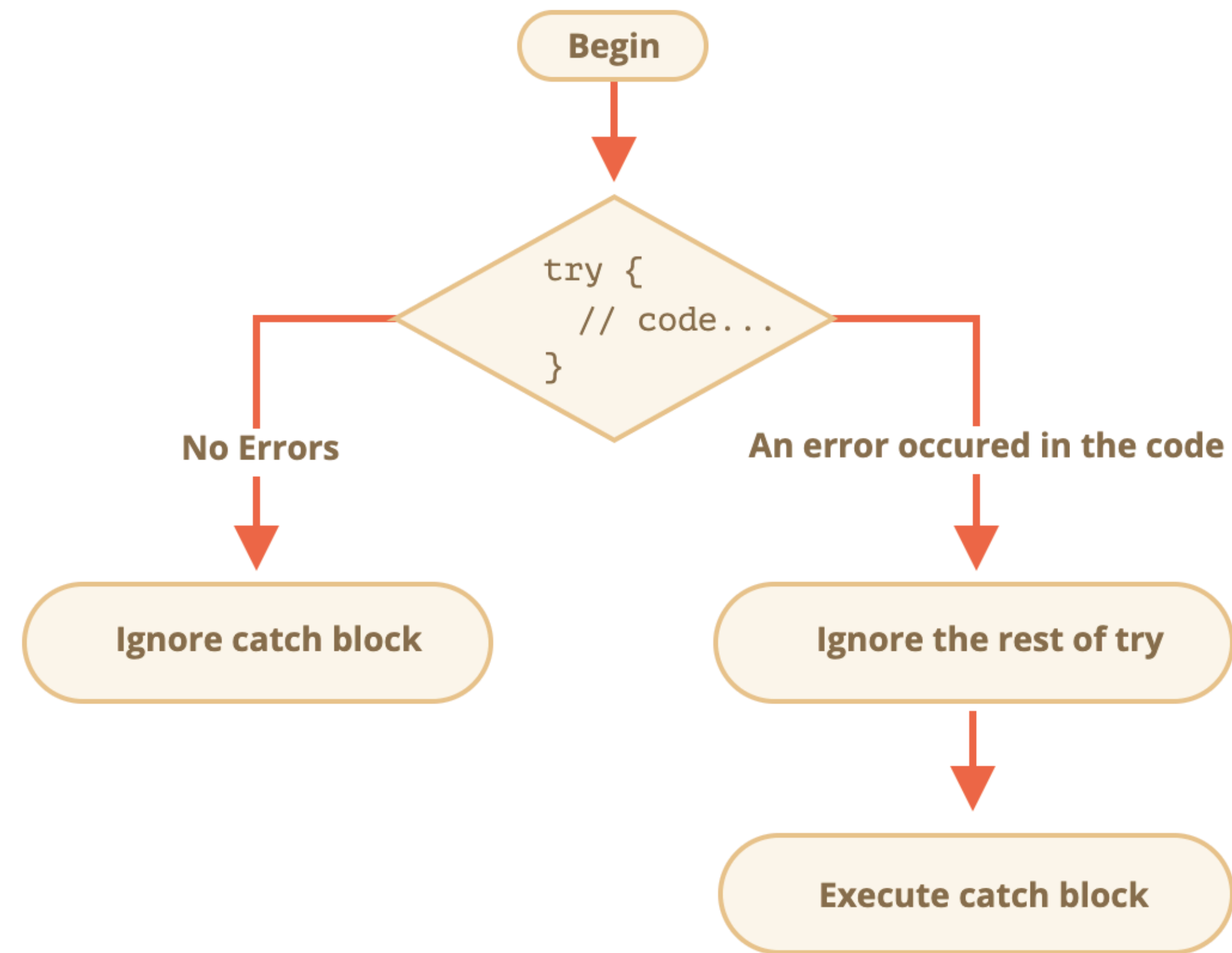
Finally

Optional enclosed part of code that will run after the try block or after the catch block

Runs every time, even if an exception is not thrown

THE "TRY...CATCH" SYNTAX

```
1 try {  
2     // code...  
3 } catch (err) {  
4     // error handling  
5 }  
6  
7  
8  
9 }
```



TRY...CATCH

```
try{
    criticalCode();
}
catch(ex){
    console.log("Got an error");
    logError(ex);
}
```

THROWING...TRY...CATCH

```
try{  
    throw "An exception that is thrown every time";  
  
}  
catch(ex){  
    console.log("Got an error");  
    logError(ex);  
}
```


TRY...CATCH...FINALLY

```
try {  
    criticalCode();  
}  
catch(ex) {  
    console.log("Got an error");  
    logError(ex);  
}  
finally {  
    console.log("Code that always will run");  
}
```

ERROR HANDLING DEMO

TRY...CATCH...FINALLY

Lecture-04-ClassCompatModule > error > JS errorhandling.js > ...

```
1  function criticalCode() {
2    throw "throwing an exception";
3  }
4
5  function logError(theException) {
6    console.log(theException);
7  }
8
9  //Try..Catch
10 console.log("\n*****Try..Catch*****\n");
11
12 try {
13   criticalCode();
14 } catch (ex) {
15   console.log("Got an error");
16   logError(ex);
17 }
18
19 //Throwing in Try..Catch
20 console.log("\n*****Throwing in Try..Catch*****\n");
21
```

```
22 try {
23   throw "An exception that is thrown every time";
24 } catch (ex) {
25   console.log("Got an error");
26   logError(ex);
27 }
28
29 //Try..Catch..Finally
30 console.log("\n*****Try..Catch..Finally*****\n");
31
32 try {
33   criticalCode();
34 } catch (ex) {
35   console.log("Got an error");
36   logError(ex);
37 } finally {
38   console.log("Code that always will run");
39 }
40 function hello() {
41   console.log("\n*****Throwing Exceptions*****\n");
42 }
```


TRY...CATCH...FINALLY

```
> node errorhandling.js
```

```
*****Throwing Exceptions*****
```

```
*****Try..Catch*****
```

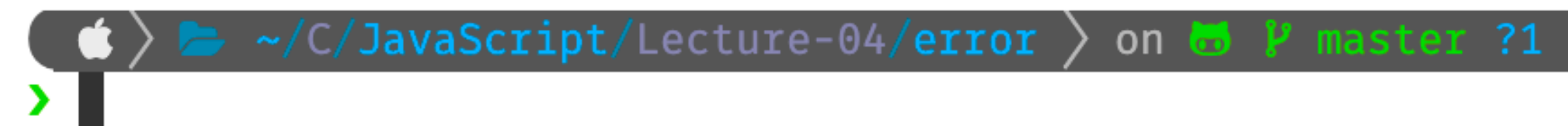
```
Got an error  
throwing an exception
```

```
*****Throwing in Try..Catch*****
```

```
Got an error  
An exception that is thrown every time
```

```
*****Try..Catch..Finally*****
```

```
Got an error  
throwing an exception  
Code that always will run
```

```
>  ~/C/JavaScript/Lecture-04/error > on  P master ?1
```

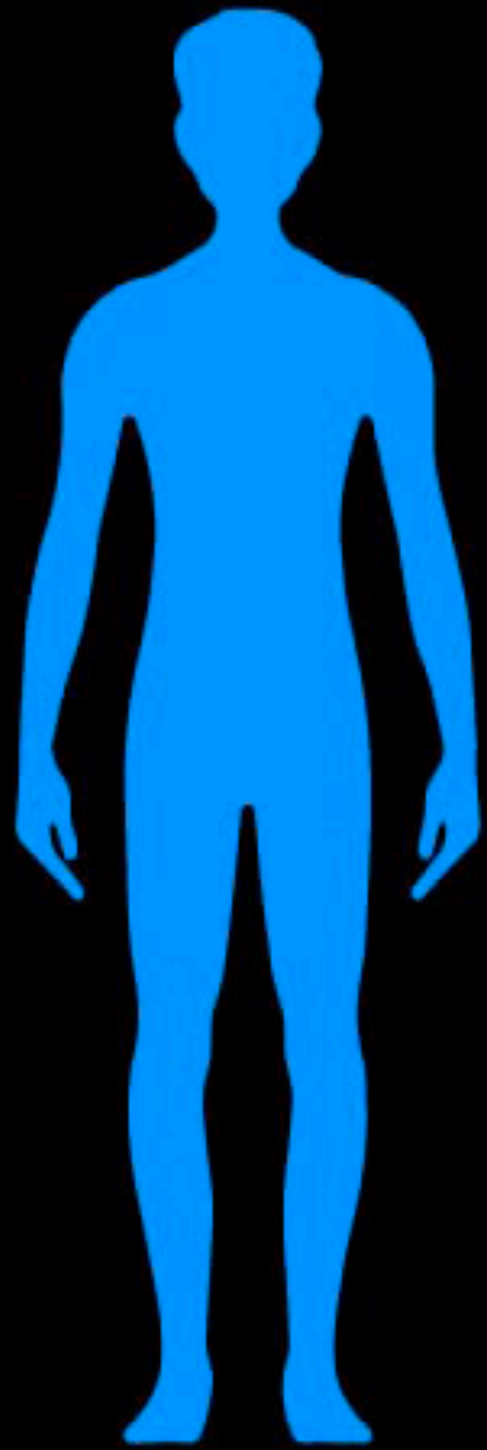
Credit: <https://sec.ch9.ms>

TRY...CATCH SAMPLE

Lecture-04-ClassCompatModule > error > <> trycash.html > html > body

```
1  <!DOCTYPE html>
2  <html>
3    <body>
4      <h2>My First Web Page</h2>
5      <p>My First Paragraph.</p>
6
7      <p id="demo"></p>
8
9      <script>
10         try {
11             lalala; // error, variable is not defined!
12         } catch (err) {
13             alert(err.name); // ReferenceError
14             alert(err.message); // lalala is not defined
15             alert(err.stack);
16             // ReferenceError: lalala is not defined at (...call stack)
17
18             // Can also show an error as a whole
19             // The error is converted to string as "name: message"
20             alert(err); // ReferenceError: lalala is not defined
21         }
22     </script>
23 </body>
24 </html>
```


BMI Ranges



Underweight
< 18.5



Healthy
18.5 - 24.9



Overweight
25 - 29.9



Obese
30 - 39.9



Severely Obese
≥ 40

CONSOLE INPUT SAMPLE

Lecture-04-ClassCompatModule > `JS` exinput.js > ...

```
1 // Program to calculate BMI
2 // BMI = weight (kg) / (height (m) * height (m))
3
4 const rl = require('readline').createInterface({
5   input: process.stdin,
6   output: process.stdout
7 });
8
9 // Prompt user for name, weight, and height
10 rl.question('Enter your name: ', (name) => {
11   rl.question('Enter your weight (in kg): ', (weight) => {
12     rl.question('Enter your height (in m): ', (height) => {
13
14       // Calculate BMI
15       let bmi = weight / (height * height);
16
17       // Display the result in 2 decimal places
18       console.log(`${name}, your BMI is ${bmi.toFixed(2)}`);
19
20       rl.close();
21     });
22   });
23 });
24
```

```
> node exinput.js
Enter your name: Anirach Mingkhwan
Enter your weight (in kg): 85
Enter your height (in m): 1.73
Anirach Mingkhwan, your BMI is 28.40
```

THANK YOU

- Class
 - Module
 - Error Handling
-